

线程同步

线程同步

1、多线程同时访问共享数据时可能会冲突

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define N_LOOP 5000
5
6 int count;
7
8 void *do_it(void *arg)
9 {
10     int i, val;
11     for(i = 0; i < N_LOOP; i++)
12     {
13         val = count;
14         printf("%x\t%d\n", (unsigned int)pthread_self(), val+1);
15         count = val + 1;
16         //usleep(500);
17     }
18     return NULL;
19 }
20
21 int main(void)
22 {
23     pthread_t tid1, tid2;
24
25     pthread_create(&tid1, NULL, do_it, NULL);
26     pthread_create(&tid2, NULL, do_it, NULL);
27
28     pthread_join(tid1, NULL);
29     pthread_join(tid2, NULL);
30
31     return 0;
32 }
```

创建两个线程，各自把count增加5000次，正常情况下最后count应该等于10000，但事实上每次运行该程序的结果都不一样。

2、线程同步的原因

- 1、共享资源，多个线程都可对共享资源操作；
- 2、线程操作共享资源的先后顺序不确定；
- 3、处理器对存储器的操作一般不是原子操作；

3.0、互斥量

mutex操作原语

```
1 pthread_mutex_t; //类型
2
3 pthread_mutex_init; //初始化锁
4
5 pthread_mutex_destroy; //销毁锁
6
7 pthread_mutex_lock; //加锁，未抢到锁的线程会被阻塞，然后线程被挂起
8
9 pthread_mutex_trylock; //尝试加锁，未抢到锁的线程会立即返回，非阻塞。
10
11 pthread_mutex_unlock; //解锁
```

在同一进程中的线程，如果加锁后没有解锁，则任何其他线程都无法再获得锁。

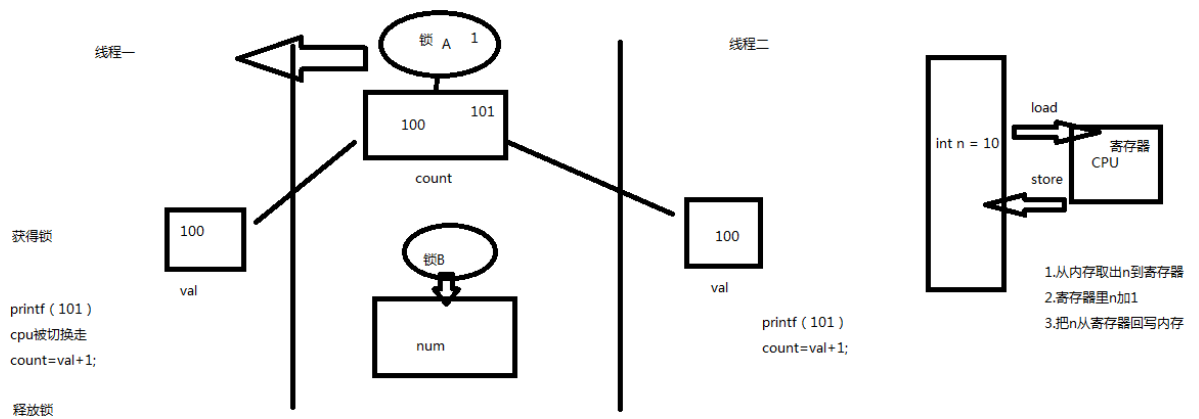
3.1、临界区（critical section）

保证在某一时刻只有一个线程能访问数据的简便办法。在任意时刻只允许一个线程对共享资源进行访问。

临界区在被释放后，其他线程可以继续抢占，并以此达到用原子方式操作共享资源的目的。

临界区是加锁后到解锁的代码块区域，称为临界区。

临界区越小越好，并发性就越好。



3.2、临界区的选定

临界区的选定尽可能的小，如果选定太大会影响程序的并行处理性能。

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define N_LOOP 5000
5
6 int count;
7
8 //设置锁
9 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
10
11 void *do_it(void *arg)
12 {
13     int i, val;
14
15     for(i = 0; i < N_LOOP; i++)
16     {
17         pthread_mutex_lock(&mutex); //加锁, 取锁1
18         //临界区
19         val = count;
```

```

20     printf("%x\t%d\n", (unsigned int)pthread_self(), val+1);
21     count = val + 1;
22     pthread_mutex_unlock(&mutex); // 解锁，释放锁
23     //usleep(500);
24 }
25 return NULL;
26 }
27
28
29 int main(void)
30 {
31
32     pthread_t tid1, tid2;
33     //pthread_mutex_init(&mutex, NULL); // 初始化锁
34
35     pthread_create(&tid1, NULL, do_it, NULL);
36     pthread_create(&tid2, NULL, do_it, NULL);
37
38     pthread_join(tid1, NULL);
39     pthread_join(tid2, NULL);
40
41     pthread_mutex_destroy(&mutex); // 销毁锁
42
43     return 0;
44 }

```

3.3、初始化锁分为两种方式

1、静态初始化锁（不需要修改属性时，可以使用）：

```

1 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

```

2、动态初始化锁：

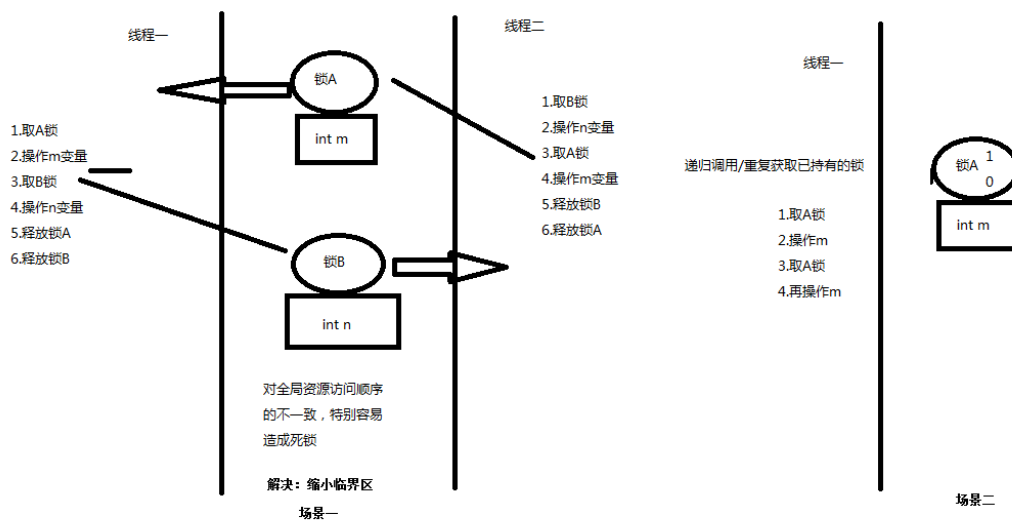
```

1 pthread_mutex_init(&mutex, NULL);

```

4、死锁

- 1、 同一个线程在拥有A锁的情况下再次请求获得A锁；
- 2、 线程1拥有A锁，请求获得B锁；线程2拥有B锁，请求获得A锁。



5、读写锁

- 1、 读共享，写独占。

多个线程可以同时获得读锁，解除了读锁，才能进行写锁。

应用在读操作多，写操作少的情况下。

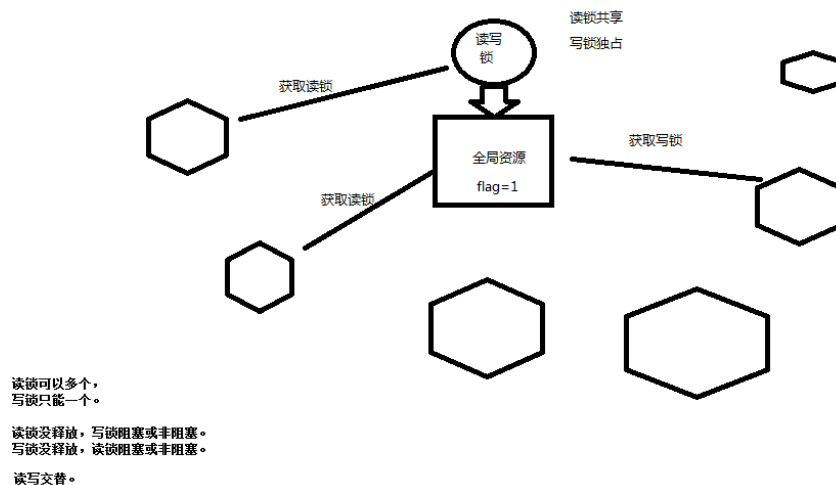
读写锁原语：

```
1 pthread_rwlock_t; //读写锁类型
2
3 pthread_rwlock_init; //初始化读写锁
4
5 pthread_rwlock_destroy; //销毁读写锁
6
7 pthread_rwlock_rdlock; //读锁，阻塞
```

```

8
9 pthread_rwlock_wrlock; //写锁，阻塞
10
11 pthread_rwlock_tryrdlock; //尝试的读锁，非阻塞
12
13 pthread_rwlock_trywrlock; //尝试的写锁，非阻塞
14
15 pthread_rwlock_unlock; //解锁

```



2、pthread_rwlock.c

```

1 #include <stdio.h>
2 #include <pthread.h>
3
4 int counter;
5 pthread_rwlock_t rwlock;
6 //3个线程不定时写同一个全局资源，5个线程不定时读同一个全局资源
7 void *th_write(void *arg)
8 {
9     int t;
10
11     while(1)
12     {
13         //写锁
14         pthread_rwlock_wrlock(&rwlock);
15         t = counter;
16         usleep(100);

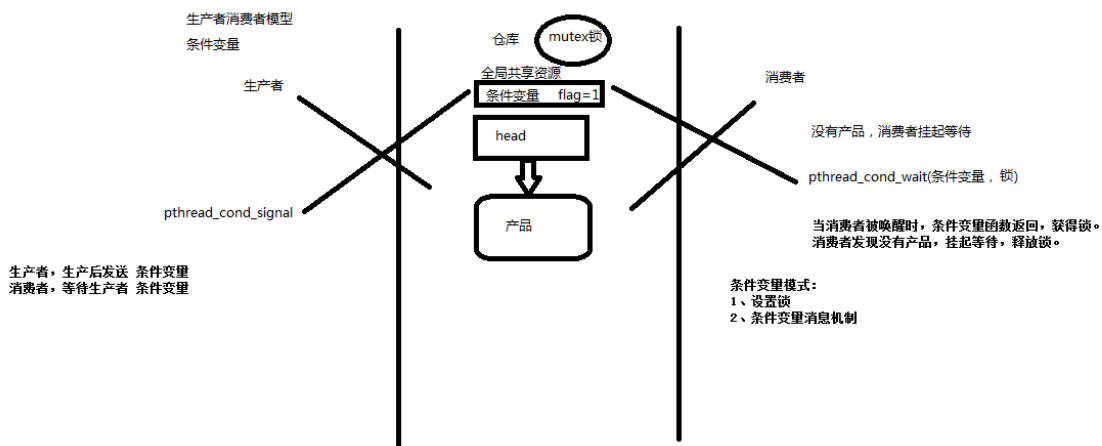
```

```
17     printf("write %x : counter = %d      ++counter = %d\n", (int)pthread_self(), t, ++counter);
18
19     pthread_rwlock_unlock(&rwlock); //解写锁
20     usleep(10000);
21
22 }
23 return NULL;
24 }
25
26 void *th_read(void *arg)
27 {
28     while(1)
29     {
30         pthread_rwlock_rdlock(&rwlock); //读锁
31         printf("*****read %x : %d\n", (int)pthread_self(), counter);
32         pthread_rwlock_unlock(&rwlock); //解读锁
33         usleep(100);
34     }
35 }
36
37 int main(void)
38 {
39     int i;
40     pthread_t tid[8];
41
42     pthread_rwlock_init(&rwlock, NULL); //初始化读写锁
43
44     for(i = 0; i < 3; i++)
45         pthread_create(&tid[i], NULL, th_write, NULL);
46
47     for(i = 0; i < 5; i++)
48         pthread_create(&tid[i+3], NULL, th_read, NULL);
49
50     for(i = 0; i < 8; i++)
51         pthread_join(tid[i], NULL);
52
53     pthread_rwlock_destroy(&rwlock); //销毁读写锁
54
55     return 0;
56 }
```

6、 条件变量

1、 条件变量原语：

```
1 pthread_cond_t; //条件变量类型
2
3 pthread_cond_init; //初始化条件变量
4
5 pthread_cond_destroy; //销毁条件变量
6
7 pthread_cond_wait; //等待挂起，对锁操作：挂起阻塞释放锁，函数返回时获得锁。
8
9 pthread_cond_timedwait; //定时等待条件变量，超时出错返回。
10
11 pthread_cond_signal; //发送条件变量信号
12
13 pthread_cond_broadcast; //广播条件变量信号
```



2、消费者和生产者模型

消费者：获得锁，判断是否有产品，如果没有则释放锁，挂起等待；

条件变量唤醒消费者，获得锁，去消费产品，然后释放锁。

生产者：获得锁，生产产品，释放锁，通知条件变量。


```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <stdlib.h>
4
5 struct msg
6 {
7     struct msg *next;
8     int num;
9 };
10
11 struct msg *head;
12
13 pthread_cond_t has_product = PTHREAD_COND_INITIALIZER; //条件变量
14 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER; //互斥锁
15
16
17 //消费者
18 void *consumer(void *p)
19 {
20     struct msg *mp;
21     for(;;)
22     {
23         pthread_mutex_lock(&lock);
24         while(head == NULL) //判断是否有产品
25             pthread_cond_wait(&has_product, &lock); //没有产品释放锁，挂起线程；返回时获得
26         mp = head;
27         head = mp->next;
28         pthread_mutex_unlock(&lock); //释放锁
29         printf("Consume %d\n", mp->num);
30         free(mp);
31         sleep(rand() % 5);
32     }
33 }
34
35 //生产者
36 void *producer(void *p)
37 {
38     struct msg *mp;
39     for(;;)
40     {
41         mp = malloc(sizeof(struct msg));
42         mp->num = rand()%1000+1;
```

```

43     printf("Produce %d\n",mp->num);
44     pthread_mutex_lock(&lock);//获取锁
45     mp->next =head;//头插法
46     head = mp;
47     pthread_mutex_unlock(&lock);//释放锁
48     pthread_cond_signal(&has_product);//发送信号量
49     sleep(rand() % 5);
50 }
51 }
52
53 int main(int argc,char *argv[])
54 {
55     pthread_t pid,cid;
56     srand(time(NULL));
57
58     pthread_create(&pid,NULL,producer,NULL);
59     pthread_create(&cid,NULL,consumer,NULL);
60
61     pthread_join(pid,NULL);
62     pthread_join(cid,NULL);
63
64     return 0;
65 }

```

7、信号量

1、信号量控制原语

```

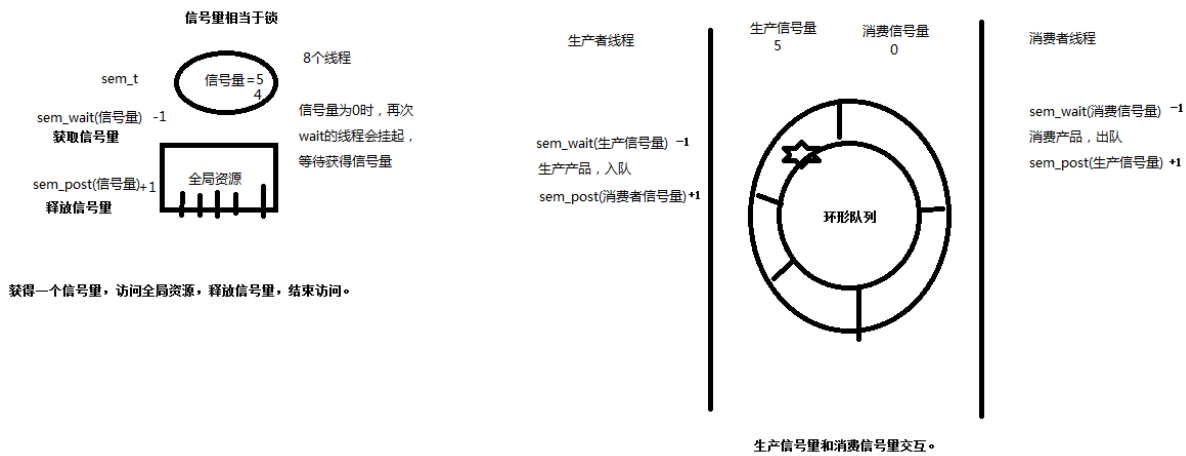
1 sem_t;//信号量的类型
2
3 sem_init;//信号量初始化（信号量：按数量计算）
4
5 sem_wait;//获取信号量--，阻塞
6
7 sem_trywait;//尝试等待信号量--，非阻塞
8
9 sem_timedwait;//定时等待信号量--，超时返回
10

```

```

11 sem_post; //释放信号量++
12
13 sem_destroy; //销毁信号量

```



消费信号量和生产信号量相互交替。

2、pthread_sem.c

```

1 #include <stdlib.h>
2 #include <pthread.h>
3 #include <stdio.h>
4 #include <semaphore.h>
5
6 #define NUM 5
7
8 int queue[NUM];
9 sem_t blank_number, product_number; //消费者和生产者
10
11 //生产者
12 void *producer(void *arg)
13 {
14     int p = 0;
15     while(1)
16     {
17         sem_wait(&blank_number); //生产信号量--, 获取
18         queue[p] = rand() % 1000 + 1;
19         printf("Produce %d\n", queue[p]);

```

```

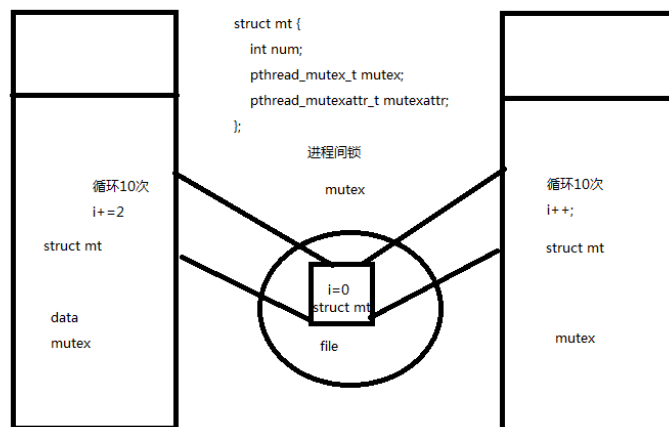
20     sem_post(&product_number); //消费信号量++, 释放
21     p = (p + 1) % NUM;
22     sleep(rand() % 5);
23 }
24 }
25
26 //消费者
27 void *consumer(void *arg)
28 {
29     int c = 0;
30     while(1)
31     {
32         sem_wait(&product_number); //消费信号量--, 获取
33         printf("Consume %d\n", queue[c]);
34         queue[c] = 0;
35         sem_post(&blank_number); //生产信号量++, 释放
36         c = (c+1)%NUM;
37         sleep(rand()%5);
38     }
39 }
40
41 int main(int argc, char *argv[])
42 {
43     pthread_t pid, cid;
44
45     //初始化信号量
46     sem_init(&blank_number, 0, NUM); //默认属性 0
47     sem_init(&product_number, 0, 0);
48     //创建线程
49     pthread_create(&pid, NULL, producer, NULL);
50     pthread_create(&cid, NULL, consumer, NULL);
51
52     pthread_join(pid, NULL); //等待回收线程
53     pthread_join(cid, NULL);
54
55     sem_destroy(&blank_number); //销毁信号量
56     sem_destroy(&product_number);
57     return 0;
58 }

```

8、进程间锁（锁的属性）

1、进程间pthread_mutex

```
1 int pthread_mutexattr_init(pthread_mutexattr_t *attr); //初始化
2
3 int pthread_mutexattr_setpshared(pthread_mutexattr_t *attr, int pshared); //设置
4
5 int pthread_mutexattr_destroy(pthread_mutexattr_t *attr); //销毁
6
7 pshared:
8 线程锁: PTHREAD_PROCESS_PRIVATE
9 进程锁: PTHREAD_PROCESS_SHARED
10 默认情况是线程锁
```



2、pthread_mutex_shared.c

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4 #include <sys/stat.h>
5 #include <sys/types.h>
6 #include <fcntl.h>
7 #include <sys/mman.h>
```

```
7 #include <sys/mman.h>
8 #include <string.h>
9
10 struct mt
11 {
12     int num;
13     pthread_mutex_t mutex; //锁
14     pthread_mutexattr_t mutexattr; //锁属性
15 };
16
17 int main(void)
18 {
19     int fd,i;
20     struct mt *mm;
21
22     pid_t pid;
23     //打开文件
24     fd = open("mt_test", O_CREAT|O_RDWR, 0777);
25     //设置文件大小
26     ftruncate(fd, sizeof(*mm));
27     //映射文件
28     mm = mmap(NULL, sizeof(*mm), PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
29     close(fd);
30     memset(mm, 0, sizeof(*mm));
31     //初始化锁的属性
32     pthread_mutexattr_init(&mm->mutexattr);
33     //设置锁的属性
34     //设置互斥对象为PTHREAD_PROCESS_SHARED共享，即可以在多个进程的线程访问；
35     //PTHREAD_PROCESS_PRIVATE为同一个进程的线程共享
36     pthread_mutexattr_setpshared(&mm->mutexattr, PTHREAD_PROCESS_SHARED);
37     //初始化锁
38     pthread_mutex_init(&mm->mutex, &mm->mutexattr);
39
40     pid = fork();
41
42     if(pid == 0)
43     {
44         for(i = 0; i < 10; i++)
45         {
46             pthread_mutex_lock(&mm->mutex); //加锁
47             (mm->num)++;
48             printf("num++:%d\n", mm->num);
49             pthread_mutex_unlock(&mm->mutex); //解锁
50             sleep(1);

```

```

51     }
52 }
53 else if(pid > 0)
54 {
55     for(i = 0; i < 10; i++)
56     {
57         pthread_mutex_lock(&mm->mutex);
58         mm->num += 2;
59         printf("num+=2:%d\n", mm->num);
60         pthread_mutex_unlock(&mm->mutex);
61         sleep(1);
62     }
63     wait(NULL);
64 }
65 pthread_mutex_destroy(&mm->mutex); //销毁锁
66 pthread_mutexattr_destroy(&mm->mutexattr); //销毁锁的属性
67 //父子均需要释放
68 munmap(mm, sizeof(*mm)); //释放映射内存
69 unlink("mt_test"); //删除文件
70
71 return 0;
72 }

```

2、文件锁

1、使用fcntl函数提供文件锁

```

1 struct flock
2 {
3     short l_type; // F_RDLCK \ F_WRLCK \ F_UNLCK
4     short l_whence; // SEEK_SET \ SEEK_CUR \ SEEK_END
5     off_t l_start; // 起始偏移量
6     off_t l_len; // 字节数
7     pid_t l_pid; // 进程pid
8     ...
9 };

```

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <sys/stat.h>
4 #include <fcntl.h>
5 #include <unistd.h>
6 #include <stdlib.h>
7
8 void sys_err(char *str)
9 {
10     perror(str);
11     exit(1);
12 }
13
14 int main(int argc, char *argv[])
15 {
16     int fd;
17     struct flock f_lock; // 文件锁
18
19     if(argc < 2)
20     {
21         printf("./a.out filename\n");
22         exit(1);
23     }
24
25     if((fd = open(argv[1], O_RDWR)) < 0)
26     {
27         sys_err("open");
28     }
29
30     // 写锁
31     f_lock.l_type = F_WRLCK;
32     // f_lock.l_type = F_RDLCK; // 读锁
33     f_lock.l_whence = SEEK_SET;
34     f_lock.l_start = 0;
35     f_lock.l_len = 0; // 0 表示整个文件加锁
36
37     // 设置写锁
38     fcntl(fd, F_SETLKW, &f_lock);
39     printf("get flock\n");
40     sleep(10);
41     // 取消锁
```



```
42     f_lock.l_type = F_UNLCK;
43     fcntl(fd, F_SETLKW, &f_lock);
44     printf("un flock\n");
45     close(fd);
46
47     return 0;
48 }
```